eBay Playwright Automation

Architecture of the test repository. 19 October 2026.

This suite drives a real browser against ebay.com. It searches for a product under a price cap, adds listings to the cart, and checks that the cart item subtotal stays inside the budget. The document below is the map of that code: which layer owns which decision, how one pytest run moves through the files, and where a new case or a new page action belongs.

1. What the suite is responsible for

The checked-in case is a guest search for shoes priced at or below 220, collecting up to five listings. That case lives in `data/test_data.json` as `shoes_under_220`. The Python test does not contain the query or the price. It reads every object in the JSON file and runs once per object.

Four business actions are required, and they all live on one class, `EbayShopService`:

1. `authenticate` — open the site as a guest, or sign in when asked.
2. `search_items_by_name_under_price` — search, apply the price filter, collect listing URLs.
3. `add_items_to_cart` — open each listing, choose variants if the page requires them, click Add to cart.
4. `assert_cart_total_not_exceeds` — read the cart item subtotal and assert it is at most price cap times number of items added.

There are no offline unit tests. Every test under `tests/` is marked `e2e` and opens ebay.com.

2. Layers

Each layer is allowed to call the one below it. Tests do not hold locators. Page objects do not decide the budget. The service does not know CSS selectors.

```
pytest
  tests/test_ebay_shopping_flow.py     one test, parametrized from JSON
  services/ebay_shop_service.py        the four business actions, Allure steps
  pages/                                 locators and clicks (page objects)
  Playwright Page                        the browser
  https://www.ebay.com
```

Supporting code sits beside that stack rather than inside it:

Path	Role
`config/settings.py`	Reads `.env` and process environment into one frozen `Settings` object.
`utils/data_loader.py`	Loads the `scenarios` array at collection time.
`utils/price_parser.py`	Turns visible price text into a float. Used by search cards and the cart.
`utils/url_utils.py`	Reduces an item link to `https://www.ebay.com/itm/<id>`.
`utils/reporting.py`	Saves a screenshot and attaches text or PNG files to Allure.
`conftest.py`	Browser launch, viewport, timeouts, failure screenshot, flow video.

3. How one test runs

`tests/test_ebay_shopping_flow.py` is the only test module. Pytest collects it like this:

1. `load_test_scenarios()` reads `data/test_data.json`. An empty or missing `scenarios` list fails collection.
2. Each object becomes one test. The node id is the scenario `id`, so the shoes case is `test_search_add_to_cart_total_within_budget[chromium-shoes_under_220]`.
3. The `ebay_shop` fixture builds an `EbayShopService` on a fresh Playwright `page` and sets the default timeout from `TIMEOUT_MS` (45 seconds).
4. The test copies `query`, `max_price`, `limit`, and `auth_mode` onto the Allure result with `allure.dynamic.parameter`. Those calls only label the report. Pytest already passed the whole scenario dict.
5. The test calls the four service actions, then asserts that at least one URL was found and that the number of URLs does not exceed `limit`. The budget assertion itself is inside the service.

> Each test gets its own browser context and its own cart. Nothing is shared between scenarios. That is what makes `pytest -n 2` safe. Live eBay often blocks a second browser from the same machine, so two workers is the practical ceiling.

4. Repository map

Path	What it contains
`tests/test_ebay_shopping_flow.py`	The e2e test. No locators.
`services/ebay_shop_service.py`	Composes page objects into the four actions. Allure `@step` decorators live here.
`pages/base_page.py`	Shared navigation, overlay dismissal, visibility helper, error-page detection.
`pages/home_page.py`	Open ebay.com and submit the header search.
`pages/search_results_page.py`	Price filter, Buy It Now, card parsing, pagination, error-page recovery.
`pages/item_page.py`	Variant selection and Add to cart.
`pages/cart_page.py`	Open the cart and read the item subtotal.
`pages/sign_in_page.py`	Two-step login. Used only when `auth_mode` is `login`.
`data/test_data.json`	External scenario injector.
`conftest.py`	Playwright fixtures and the Allure video hook.
`pytest.ini`	Markers, Allure directory, video, screenshots, traces, JUnit path.
`config/settings.py`	Environment settings. Process env wins over `.env`.
`.env.example`	Documented defaults. Copy to `.env` for a local run. `.env` is gitignored.
`scripts/build_run_report.py`	Builds the single checked-in HTML report, `run-report.html`.
`scripts/docker-entrypoint.sh`	Starts pytest under xvfb unless `HEADLESS` is true.
`Dockerfile`	`mcr.microsoft.com/playwright/python:v1.64.0-noble`, plus xvfb and Allure 2.32.
`docker-compose.yml`	Services `tests`, `allure`, and `allure-ui`.
`.github/workflows/tests.yml`	The live suite on GitHub-hosted runners.
`.claude/skills/add-ebay-tests/SKILL.md`	The one shared rule for adding tests. Cursor and Claude Code both load this file.

Generated output stays out of git: `allure-results/`, `allure-report/`, `reports/`, and `test-results/`. The exception is `run-report.html` in the repository root, which is a compressed example of one finished run.

5. Page objects

`BasePage` holds the Playwright `page` and the settings object. Every other page class inherits it.

- `goto` opens a URL with `domcontentloaded`, waits 800 ms, then calls `dismiss_overlays`.
- `dismiss_overlays` closes the shipping-address lightbox first, then tries a short list of cookie, survey, and dialog close buttons. A failure on one button does not fail the test.

- `visible` and `click_first_visible` walk a list of fallback selectors and click the first one that appears. Locators are written as ordered fallbacks because eBay's markup changes.
- `is_error_page` is true when the document title contains "Error Page". eBay serves that page when it blocks the client: headless Chrome, a datacenter IP, or a filter submit it rejects.
- `wait_for_network_idle` waits up to 8 seconds, then continues on DOM ready. eBay keeps polling, so the idle wait often uses the full timeout.

HomePage

`open` goes to `EBAY_BASE_URL` and reloads once if the title contains "Error Page". `search` dismisses overlays, types into `#gh-ac` (with name and generic search-input fallbacks), and clicks `#gh-search-btn`. It does not call `dismiss_overlays` between filling the box and clicking Search, because that pass was sitting on the homepage while the suggestion list was open. After submit it waits for a `/sch/` URL, dismisses overlays again, and waits up to 20 seconds for result cards.

SearchResultsPage

Listing cards are found with an XPath that requires a real `/itm/` link, so promo tiles are skipped. For each card the page reads the link and the price node, canonicalizes the URL, and keeps the card only when `parse_listing_price` returns a number at or below the cap. A range such as "$50 to $250" uses the higher number, so a listing that can exceed the budget is not collected. A single price, including a sale price next to an original price, uses the first amount.

`apply_max_price_filter` fills the left-rail maximum field and submits it. If that submit lands on the error page, the page goes back and the in-code price check still applies. If the inputs are missing, filtering is done only in code. `apply_buy_it_now_filter` checks the Buy It Now box when it is present, so the later Add to cart action has a listing that can be purchased. `go_to_next_page` walks pagination until `limit` URLs are collected or `MAX_SEARCH_PAGES` (default 8) is reached. `recover_from_error` leaves the error page by clicking the homepage link, which is more reliable than staying on the blocked response.

ItemPage

`add_to_cart` dismisses overlays, picks a random in-stock value for each native `<select>` and each unresolved listbox (size, color, width), then clicks the first visible Add to cart control. Selectors are tried in order: the classic button id, the current buy-box test id, and a secondary call-to-action. If the click reveals that a variant is still required, selection runs again and the button is clicked once more. The confirmation lightbox is closed. `RANDOM_SEED` makes those picks repeatable. An empty seed means a new sequence each run. A listing with no Add to cart control returns false and is skipped.

CartPage

`open` clicks the header cart link when it is visible, otherwise it goes straight to `https://cart.ebay.com/`. `read_item_subtotal` prefers `[data-test-id='ITEM_TOTAL']`, then subtotal test ids, then older class names, then a row whose text looks like "items (n)". Shipping and tax rows are skipped. The assertion uses the item subtotal so shipping and import charges do not fail a cart that is inside the item budget.

SignInPage

Login is a two-step form: user id, continue, password, submit. If the browser is still on a sign-in URL afterwards, the method raises. Captcha and 2FA are the usual reason. The default scenario never calls this page.

6. EbayShopService

The service is constructed with one `page`. It creates the five page objects and one `random.Random` from the settings seed. Public methods are Allure steps, so the report shows “Authenticate”, “Search”, “Add items to cart”, and “Assert cart total” as a tree. Adding each listing is a nested step.

authenticate(mode). The test always passes the scenario’s `auth_mode`, which defaults to `guest` inside the test. `EBAY_AUTH_MODE` is used only when `authenticate()` is called with no mode. Guest opens the homepage when the current URL is not ebay.com or is an error page. Login calls `SignInPage.login` with `EBAY_EMAIL` and `EBAY_PASSWORD`.

search_items_by_name_under_price(query, max_price, limit). Search, and if the error page appears, recover and search once more. Apply the max-price filter, then Buy It Now. Screenshot the results. Collect cards, paginate if fewer than `limit` URLs were found, and attach the URL list to Allure. The last search URL is remembered so each item can return to that results page.

add_items_to_cart(urls). For each URL: open the item, try Add to cart, screenshot, and if it succeeded record the URL. Then go back to the saved search URL. If every listing is skipped, the method raises. The test fails only when nothing could be added.

assert_cart_total_not_exceeds(budget_per_item, items_count). Opens the cart, screenshots it, reads the subtotal, and asserts `subtotal <= budget_per_item * items_count` with a one-cent tolerance. For the shoes scenario that threshold is 220 times the number of items actually added, which is 1100 when all five succeed. The numeric cap is compared to the price the site displays. There is no currency conversion. A machine whose eBay locale shows ILS still applies 220 to those displayed numbers.

7. Scenarios

`data/test_data.json` is the external scenario injector. Docker mounts `./data` read-only, so the host file is what the container runs. Adding another search is a new object in this file. The test module stays as it is.

Field	Required	Meaning
`id`	no	Name in pytest and Allure. Falls back to `query`.
`query`	yes	Text typed into eBay search.
`max_price`	yes	Numeric cap, in the site’s displayed currency.
`limit`	no	How many listing URLs to collect. Default 5.
`auth_mode`	no	`guest` (default) or `login`.

The shoes object is `id=shoes_under_220`, `query=shoes`, `max_price=220`, `limit=5`, `auth_mode=guest`.

8. Configuration and the browser

`get_settings()` is cached. A variable already set in the shell, Compose, or GitHub Actions overrides `.env`.

Variable	Default	Effect
`EBAY_BASE_URL`	`https://www.ebay.com`	Site the suite opens.
`EBAY_AUTH_MODE`	`guest`	Default only when `authenticate()` is called with no mode.
`EBAY_EMAIL` / `EBAY_PASSWORD`	empty	Login mode.
`HEADLESS`	`false`	`true`, `1`, `yes`, or `on` hide the window. eBay often blocks headless Chrome.
`SLOW_MO_MS`	`0`	Playwright slow-motion delay.
`TIMEOUT_MS`	`45000`	Default page and navigation timeout.
`BROWSER_CHANNEL`	empty	Set to `chrome` to drive the installed Google Chrome. Docker and CI force this empty.
`RANDOM_SEED`	empty	Seed for size and color picks.
`MAX_SEARCH_PAGES`	`8`	Search-result pages walked while collecting listings.

`conftest.py` sets a 1440×900 viewport, locale `en-US`, timezone `America/New_York`, and an Accept-Language header. Launch flags include `--disable-blink-features=AutomationControlled`. Docker and CI also pass `--no-sandbox` and `--disable-gpu`. The same file writes `allure-results/environment.properties` so the Allure overview shows the base URL, auth mode, and headless flag.

9. Reporting

`pytest.ini` always passes:

```
--alluredir=allure-results
--video=on
--screenshot=only-on-failure
--tracing=retain-on-failure
--junitxml=reports/junit.xml
```

Flow video. `--video=on` records a `.webm` for every test that opens a browser. The file is only complete after the browser context closes, which happens during teardown. pytest-playwright then stores the path on the report. `conftest.py` runs its `pytest_runtest_makereport` hook first on the way in and last on the way out, so it sees that path and attaches it to the still-open Allure test as `recording` (`video/webm`). `pytest --video=off` overrides the ini file and skips recording.

Screenshots. The service attaches a full-page PNG after search, after each add-to-cart attempt, and on the cart. Files are also written under `reports/screenshots/`. On a failed test call, `conftest.py` attaches a `failure` screenshot and the page URL.

Allure HTML. `allure generate allure-results -o allure-report --clean` builds the static report. `allure serve allure-results` opens it without writing that folder. Both raw folders are gitignored.

Checked-in example. `python scripts/build_run_report.py` keeps the newest Allure result per test, compresses its screenshots, and writes one self-contained `run-report.html` at the repository root. That

file does not embed the `.webm` recordings.

JUnit XML is written to `reports/junit.xml` for anything that consumes JUnit rather than Allure.

10. How the suite is executed

Local

Python 3.10 or newer, a virtualenv, `pip install -r requirements.txt`, and `playwright install chromium`. Direct dependencies are pytest 9.1.1, pytest-playwright 0.10.0, playwright 1.64.0, allure-pytest 2.16.2, pytest-xdist 3.8.0, and python-dotenv. The Playwright version matches the Docker image tag.

```
pytest
pytest -m e2e
pytest -n 2 -m e2e
pytest --video=off
```

`-n` comes from pytest-xdist and is optional. The default run is serial. `-n auto` starts one worker per CPU. For this suite, `-n 2` is the setting that matches two live browsers.

Docker

`docker compose run --rm tests` builds `ebay-playwright-automation` and runs pytest as `pwuser`. The entrypoint wraps pytest in xvfb when `HEADLESS` is not true, because eBay blocks classic headless Chrome and the container has no physical display. Compose publishes `allure-results`, `allure-report`, and `reports` to the host, and mounts `./data` read-only. `docker compose up allure-ui` serves the report on port 5050. `docker compose run --rm allure` writes the static HTML into `allure-report/`.

GitHub Actions

`.github/workflows/tests.yml` runs on push to `main`, on pull requests, and on manual dispatch. There is one job, `e2e`. It installs Python 3.12, the requirements, Chromium, and xvfb, then runs `pytest -m e2e` with `HEADLESS=false` inside xvfb at 1440×900. The job is `continue-on-error` because ebay.com often blocks GitHub-hosted IP addresses. Allure results and the `reports` folder are uploaded as the `e2e-allure-results` artifact even when the job fails. A second push to the same ref cancels the in-progress run.

11. What the code does when eBay pushes back

- Title contains “Error Page” on the homepage: reload once.
- The same page after search: click the link back to ebay.com, or `goto` the base URL, then search again.
- The price-filter submit lands on the error page: `go_back`, and keep filtering cards in code.
- A listing has no Add to cart control, or a required variant cannot be chosen: skip it and continue. Fail only if the added list is empty.
- Login remains on the sign-in URL: raise, and tell the caller to use guest mode.
- The cart total cannot be parsed: raise `AssertionError`.

Headed Chromium is the default for this reason. Docker uses a virtual display rather than Playwright’s headless mode. A datacenter IP can still receive the error page even in headed mode. When that

happens, the same command on a normal desktop is the reliable run.

12. Where a change belongs

The shared rule for adding tests is `.claude/skills/add-ebay-tests/SKILL.md`. Cursor loads project skills from `.claude/skills/`, and Claude Code loads the same path, so there is one file.

- Another query or another budget: a new object in `data/test_data.json`. Do not copy the test.
- A new click or a new locator: a method on the matching class in `pages/`, using `BasePage` helpers. Before writing a locator, read a live snapshot with the Playwright MCP (`npx -y @playwright/mcp@latest`) rather than inventing a selector.
- A new business action the test should call as one step: a method on `EbayShopService` with `@allure.step`.
- A new test file: thin, marked `@pytest.mark.e2e`, inputs from JSON. Do not add an offline test.

Assertions belong on outcomes: listing URLs were collected, items were added, the cart subtotal is within the threshold. Creating a locator is not an assertion. Do not add `time.sleep`. The waits already on `BasePage` are the ones the site is given.